

PROJECT PROPOSAL

2D Robot Path Planning using Trajectory Optimization (MPC)

Course: Numerical Optimization

Professor: Prof. Dr. Izudin Džafić

Student: Emin Hadžiabdić

Academic Year: 2025/26

Student ID: 19960

Sarajevo, May 20th 2026

Table of Contents

The Core Idea.....	2
How it Maps to the Lectures.....	2
The Objective Function (Lecture 6).....	2
Equality Constraints & Physics (Lecture 3 & 4).....	2
Inequality Constraints & Obstacles (Lecture 6).....	3
Matrix Structures & GUI Implementation.....	3
Code Base & Adaptation Strategy.....	3

The Core Idea

For this project, I want to tackle 2D robot path planning, but instead of using a standard grid-search algorithm like A* (A star), I want to treat it entirely as a continuous mathematical optimization problem.

The goal is to find the best possible path for a mobile robot to move from Point A to Point B over a fixed timeline ($t = 1$ to N). "Best possible" means finding a trajectory that avoids obstacles, respects the robot's physical limits (like max speed and acceleration), and keeps the movement smooth to save energy.

How it Maps to the Lectures

This problem directly applies several key concepts we covered in class, specifically focusing on handling constraints and structuring large systems of equations:

The Objective Function (Lecture 6)

I will set up a **Quadratic Programming (QP)** cost function.

It will penalize two things:

1. How far the robot is from the target.
2. How aggressive the control changes are.

The goal is to minimize this cost over the entire timeline.

Equality Constraints & Physics (Lecture 3 & 4)

The robot has to obey physics - its position at time step $t+1$ depends strictly on its position, velocity, and steering angle at time step t .

When you chain these equations together over a long timeline (e.g., 50 time steps), it creates a massive system of linear equations. Because step 50 has nothing to do with step 2, the global constraint matrix is highly structured and sparse. This is a perfect opportunity to use the **Sparse LU factorization** concepts from Lecture 3.

Inequality Constraints & Obstacles (Lecture 6)

The robot cannot exceed its maximum speed, and it must maintain a minimum safe distance from any obstacle center:

$$(x_t - x_{\text{obs}})^2 + (y_t - y_{\text{obs}})^2 \geq R^2$$

Because obstacle avoidance introduces non-linear inequalities, the problem becomes non-convex. I plan to handle this iteratively using **Sequential Quadratic Programming (SQP)** or a **Lagrange-Newton** approach to evaluate the **KKT conditions** at each step.

Matrix Structures & GUI Implementation

Dense Matrices: I will use these for local state vectors (storing individual (x, y) coordinates, speeds, and heading angles) and for the tuning weights in the objective function.

Sparse Matrices: These will be used to construct the massive block-diagonal matrix that tracks the physics constraints across the entire timeline, keeping the calculations efficient.

NatID GUI Integration: I plan to build an interactive 2D simulation interface. It will render the starting point, the target, and any obstacles. The GUI will draw the optimized path as a smooth line connecting the time steps, and if I move the obstacles around using GUI controls, the solver should update and re-draw the path in real-time.

Code Base & Adaptation Strategy

Instead of writing a complex MPC framework completely from scratch, I plan to adapt the core mathematical loops from existing open-source references. Specifically, I have found two repositories on GitHub that match this scope perfectly:

1. [jayshah19949596/Model-Predictive-Control-Project](https://github.com/jayshah19949596/Model-Predictive-Control-Project) (A C++ MPC loop used for tracking 2D state configurations).
2. [DMaroo/mpc_path_planner](https://github.com/DMaroo/mpc_path_planner) (A clean, minimal C++ project focusing directly on obstacle avoidance).

My main focus will be stripping away any external dependencies these repositories use, refactoring their internal math to run entirely on the custom **Dense** and **Sparse** matrix structures required for this course, and then piping the output coordinates into the **NatID framework** for the visualization.